

DevOps. Automatización y monitorización de infraestructuras

Unidad 02. Introducción a la automatización con Vagrant



Autor: Sergi García Barea

Actualizado Mayo 2026



Licencia



Reconocimiento – NoComercial - CompartirIgual (BY-NC-SA):

No se permite un uso comercial de la obra original ni de las posibles obras derivadas, la distribución de las cuales se debe hacer con una licencia igual a la que regula la obra original.

Unidad 02. Introducción a la automatización con Vagrant	2
1. 💡 Introducción	2
2. 🌐 Arquitectura de red: el bridge virtual	2
3. 💻 Impacto en el hardware y kernel de la virtualización	3
4. 🧩 Fundamentos de Vagrant e Infraestructura como Código (IaC)	4
5. 🛠️ Comandos esenciales de Vagrant	4
5.1 Gestión de Imágenes Base (vagrant box)	4
5.2 El Despliegue Inicial (vagrant up)	5
5.3 Acceso y monitorización (vagrant ssh y vagrant status).	5
5.4 Reconfiguración y aplicación de Cambios (vagrant provision y vagrant reload)	6
5.5 El apagado y la limpieza (vagrant halt y vagrant destroy)	6
6. 📄 El fichero Vagrantfile	6
6.1 Ejemplo 1: Configuración monolítica (Servidor Único)	7
6.2 Ejemplo 2: Configuración multi-nodo (Entorno de red aislado)	8
6.3 Ejemplo 3: Integración con red física (Bridge Dedicado)	8
7. ★ Buenas prácticas y principios de diseño para un Vagrantfile eficiente	9
8. 📖 Webgrafía	11

UNIDAD 02. INTRODUCCIÓN A LA AUTOMATIZACIÓN CON VAGRANT

1. INTRODUCCIÓN

En el ecosistema moderno de IT, la capacidad de recrear entornos exactos es una necesidad crítica. La virtualización nos permite ejecutar múltiples sistemas operativos sobre un único hardware físico, pero la gestión manual de estas máquinas es ineficiente y propensa al error humano.

La automatización entra en juego para resolver el problema de la **deriva de configuración** (cuando dos máquinas que deberían ser iguales terminan siendo distintas debido a cambios manuales). Al tratar la infraestructura como si fuera software, podemos versionar, compartirla y garantizar que el entorno de desarrollo sea un reflejo fiel del entorno de producción.

2. ARQUITECTURA DE RED: EL BRIDGE VIRTUAL

Uno de los pilares de la virtualización es cómo conectamos las máquinas entre sí y con el mundo exterior. En este modelo, optamos por la creación de un **punto de red (Bridge)** dedicado.

¿Por qué usar un Bridge dedicado (ej. LabCE FIRE)?

- **Aislamiento y control:** Al crear un bridge específico (como el segmento 172.28.0.0/24), generamos un switch virtual independiente dentro del host. Esto evita colisiones con otras redes locales y nos permite asignar IPs estáticas sin depender del DHCP de la oficina o casa.
- **Simulación de producción:** A diferencia del modo NAT (que oculta la VM tras la IP del host), el modo Bridge permite que las VMs se vean entre sí como si estuvieran conectadas a un switch físico real.
- **Persistencia:** La red se define a nivel de kernel en el host, permitiendo que las máquinas virtuales se "enganchen" y "desenganchen" de una infraestructura de red que permanece estable.

Modo de Red	Visibilidad VM -> Host	Visibilidad Host -> VM	IP Propia en la Red
NAT	✓ Sí	✗ No (requiere túneles)	✗ No (IP compartida)
Host-Only	✓ Sí	✓ Sí	✓ Sí (privada)
Bridge (Puente)	✓ Sí	✓ Sí	✓ Sí (pública/segmento)

Los comandos básicos para crear y configurar el Bridge dedicado (LabCEFIRE) en Linux son:

Comandos de Configuración:

1. Cargar módulo dummy (necesario para forzar estado UP):

```
sudo modprobe dummy
```

2. Creación del Bridge:

```
sudo ip link add name LabCEFIRE type bridge
```

3. Asignación de IP (Gateway del Host):

```
sudo ip addr add 172.28.0.1/24 dev LabCEFIRE
```

4. Creación de la interfaz dummy:

```
sudo ip link add cefire-dummy type dummy
```

5. Asociar la interfaz dummy al Bridge:

```
sudo ip link set cefire-dummy master LabCEFIRE
```

6. Activar la interfaz dummy:

```
sudo ip link set cefire-dummy up
```

7. Activar el Bridge:

```
sudo ip link set LabCEFIRE up
```

Comandos de Destrucción (Limpieza):

1. Eliminar la interfaz dummy:

```
sudo ip link delete cefire-dummy
```

2. Bajar el estado del Bridge:

```
sudo ip link set LabCEFIRE down
```

3. Destruir el Bridge:

```
sudo ip link delete LabCEFIRE
```

3. IMPACTO EN EL HARDWARE Y KERNEL DE LA VIRTUALIZACIÓN

La virtualización no es "gratis"; tiene un coste físico real en el sistema anfitrión:

- **Memoria RAM:** Al reservar memoria para una VM, el Kernel del host marca esas páginas como reservadas. Si se reserva demasiado, el host entra en Thrashing (intercambio masivo a disco), degradando el rendimiento global.
- **CPU y Ring 0:** VirtualBox utiliza módulos de kernel para permitir que la VM ejecute código privilegiado directamente en la CPU. Esto es vital para el rendimiento, pero

requiere que las tecnologías VT-x/AMD-V estén activas.

- **Modo promiscuo en red:** Al usar un Bridge, la tarjeta de red del host debe operar procesando tramas Ethernet que no van dirigidas a su propia IP, lo que genera una carga de interrupciones de CPU (irq) adicional.

⚠ **Nota:** Nunca reserves más del 80% de la RAM física del host para máquinas virtuales. Si el host se queda sin memoria, el kernel activará el *Out Of Memory Killer (OOM Killer)* y destruirá el proceso de tu VM más pesada para salvarse a sí mismo.

4. 🧩 FUNDAMENTOS DE VAGRANT E INFRAESTRUCTURA COMO CÓDIGO (IAC)

Vagrant es una herramienta de orquestación diseñada para crear y configurar entornos de desarrollo completos, reproducibles y portátiles. Su objetivo es acabar con el clásico "en mi máquina funciona" mediante la adopción de un enfoque de Infraestructura como Código (IaC) de tipo declarativo.

A diferencia de un enfoque imperativo (donde das órdenes paso a paso y te arriesgas a fallos a mitad de proceso), con Vagrant defines el estado final deseado en un único archivo: el Vagrantfile. Este archivo es la Única Fuente de Verdad; si no está descrito ahí, no existe en la infraestructura.

Aunque herramientas como Docker son excelentes para microservicios efímeros (aislando procesos mediante espacios de nombres), Vagrant brilla cuando necesitamos emular hardware real y persistencia, haciéndolo ideal para laboratorios de redes, kernel hacking o administración de sistemas pura. Su arquitectura se apoya en tres grandes pilares:

- **El Provider (Abstracción de hardware):** Traduce las órdenes de Vagrant al hipervisor subyacente (como VirtualBox) para provisionar los recursos físicos virtuales.
- **La Box (Artefacto inmutable):** Es la imagen base (un disco y metadatos) a partir de la cual Vagrant crea una copia ligera (diferencial) para tu proyecto actual, ahorrando espacio en disco.
- **El Provisioner (Ciclo de vida):** Una vez que la máquina arranca y tiene red, Vagrant inyecta una clave SSH para tomar el control y ejecutar scripts que configuran automáticamente el software y los servicios necesarios.

5. 🛠 COMANDOS ESENCIALES DE VAGRANT

Para dominar la infraestructura como código con Vagrant, es fundamental entender que los comandos en la terminal no son simples "botones de encendido y apagado", sino desencadenantes de un ciclo de vida complejo. Cuando interactuamos con la herramienta, estamos enviando instrucciones precisas al hipervisor subyacente. A continuación, desglosamos los comandos principales y lo que realmente ocurre "bajo el capó" cuando los ejecutamos.

5.1 Gestión de Imágenes Base (vagrant box)

En Vagrant no usamos ISOs, usamos boxes (imágenes inmutables preconfiguradas). Cuando haces un up, si no tienes la box, Vagrant la descarga de Vagrant Cloud (el equivalente a Docker Hub). Estas imágenes no se guardan en la carpeta de tu proyecto, sino en un registro global de tu sistema (ej. ~/.vagrant.d/boxes), permitiendo crear 10 máquinas distintas compartiendo la

misma imagen base para ahorrar disco. En la Práctica:

- **vagrant box add debian/bookworm64** → Descarga la imagen base manualmente a tu caché global.
- **vagrant box list** → Muestra qué imágenes tienes descargadas en tu disco duro y para qué hipervisor.
- **vagrant box update** → Comprueba si hay parches de seguridad para tus boxes base y los descarga.
- **vagrant box remove debian/bookworm64** → Borra la imagen de la caché global para liberar espacio (no afecta a las VMs que ya estén creadas y corriendo).

5.2 El Despliegue Inicial (vagrant up)

Este es el motor de arranque de toda la orquestación. Al ejecutar este comando, Vagrant no se limita a encender una máquina; primero lee detenidamente el Vagrantfile para entender el estado deseado. Acto seguido, se comunica con el proveedor (como VirtualBox) para reservar la memoria RAM y la CPU, monta las carpetas compartidas entre el host y la máquina virtual, y arranca el sistema operativo. Una vez que el kernel de la VM está listo y el servicio de red responde, Vagrant inyecta dinámicamente una clave SSH privada para tomar el control y ejecutar, de forma secuencial, los scripts de aprovisionamiento que hayamos definido.

En la Práctica:

- **vagrant up** → Levanta todas las máquinas del entorno.
- **vagrant up webserver** → Levanta solo el nodo "webserver" (ahorra RAM).
- **vagrant up --provision** → Levanta y fuerza la ejecución de los scripts de instalación.

5.3 Acceso y monitorización (vagrant ssh y vagrant status).

Una vez que la infraestructura está levantada, necesitamos interactuar con ella. El comando `vagrant ssh` no es un protocolo propietario; es simplemente un envoltorio (wrapper) que invoca al cliente SSH estándar de tu sistema, pasándole automáticamente la ruta hacia la clave privada generada y el puerto mapeado correcto. Por su parte, `vagrant status` actúa como nuestro panel de telemetría. Al lanzarlo, Vagrant consulta el archivo de estado local y pregunta al hipervisor si el identificador único (UUID) de esa máquina sigue registrado y en ejecución, devolviéndonos el estado real de la topología.

En la Práctica:

- **vagrant ssh client** → Abre una sesión interactiva dentro del nodo "client".
- **vagrant ssh webserver -c "sudo systemctl status nginx"** → Entra, ejecuta un diagnóstico de Nginx y sale automáticamente.
- **vagrant status** → Muestra el estado de las máquinas en la carpeta actual.
- **vagrant global-status** → Escanea todo tu PC buscando máquinas Vagrant huérfanas o corriendo en segundo plano.

5.4 Reconfiguración y aplicación de Cambios (vagrant provision y vagrant reload)

A medida que la infraestructura evoluciona, a menudo necesitamos hacer ajustes. Aquí es vital distinguir entre dos escenarios. Si solo hemos modificado un script de instalación (por ejemplo, actualizando la configuración de Nginx), usamos `vagrant provision`. Esto inyecta los nuevos cambios en caliente sobre la máquina en marcha, sin reiniciar el hardware. Sin embargo, si modificamos la "arquitectura física" en el Vagrantfile (como asignarle más memoria RAM, cambiar la IP del Bridge o añadir una redirección de puertos), necesitamos usar `vagrant reload`. Este comando realiza un apagado ordenado, reconfigura el hardware virtual a través del hipervisor y vuelve a encender la máquina.

En la Práctica:

- **vagrant provision** → Ejecuta los scripts en caliente sobre las máquinas encendidas.
- **vagrant reload** → Apaga y enciende la máquina (útil si se queda "congelada").
- **vagrant reload --provision** → El combo estrella. Reinicia aplicando nuevos cambios de hardware y acto seguido ejecuta los scripts.

5.5 El apagado y la limpieza (vagrant halt y vagrant destroy)

La gestión responsable de los recursos de nuestro equipo anfitrión exige saber cómo detener la infraestructura. Cuando usamos `vagrant halt`, Vagrant envía una señal ACPI de apagado ordenado (SIGTERM) al sistema operativo invitado, permitiendo que cierre sus procesos y guarde datos en el disco duro virtual, el cual persiste intacto. Por el contrario, cuando un entorno de pruebas ya no es necesario o se ha corrompido, recurrimos a `vagrant destroy`. Este comando es destructivo: borra la máquina virtual del hipervisor y elimina sus discos duros virtuales, liberando todo el espacio. Lo único que sobrevive es nuestro Vagrantfile, listo para levantar una réplica exacta e inmaculada la próxima vez que hagamos un up.

En la Práctica:

- **vagrant halt** → Apaga ordenadamente toda la infraestructura.
- **vagrant halt webserver -f** → Tirón de cable. Fuerza el apagado inmediato sin esperar al SO.
- **vagrant destroy** → Borra las máquinas, pidiendo confirmación (y/n).
- **vagrant destroy -f** → Destruye todo sin hacer preguntas (ideal para scripts de limpieza automática).

6. EL FICHERO VAGRANTFILE

Si Vagrant es el "director de orquesta", el Vagrantfile es la partitura donde queda escrita toda la infraestructura. Es un archivo de texto que define no sólo qué sistema operativo usar, sino cómo se debe comportar el hardware virtual y qué software debe estar listo al arrancar.

El Vagrantfile utiliza una sintaxis basada en Ruby. No es necesario ser programador para editarlo, pero es vital respetar su estructura de bloques. Todo el archivo es esencialmente un gran bloque de configuración que encierra definiciones más pequeñas para cada máquina virtual.

Para entender cualquier Vagrantfile, debemos identificar estas directivas principales:

Directiva	Que representa
<code>config.vm.box</code>	El "molde" o imagen base del sistema operativo.
<code>config.vm.network</code>	Las "tarjetas de red" (Puertos redirigidos, redes privadas o bridges).
<code>config.vm.provider</code>	El "hipervisor" (ajustes de RAM, CPU y nombre en VirtualBox).
<code>config.vm.provision</code>	Los "instaladores" (scripts que configuran el software).
<code>config.vm.synced_folder</code>	Las "carpetas compartidas" entre tu PC y la VM.

A continuación comentamos algunos ejemplos de Vagrantfile

6.1 Ejemplo 1: Configuración monolítica (Servidor Único)

Este es el ejemplo más común para un desarrollador que necesita un entorno de pruebas rápido. Define una sola máquina con un puerto abierto para ver una web.

```
# -*- mode: ruby -*-
# vi: set ft=ruby :

Vagrant.configure("2") do |config|
  # 1. Definimos la imagen base
  config.vm.box = "ubuntu/focal64"

  # 2. Redirección de puertos: El puerto 80 de la VM se verá en el 8080 del Host
  config.vm.network "forwarded_port", guest: 80, host: 8080

  # 3. Ajustes de Hardware (Provider)
  config.vm.provider "virtualbox" do |vb|
    vb.memory = "1024" # 1GB de RAM
    vb.cpus = 1         # 1 Núcleo de CPU
    vb.name = "mi-servidor-unico"
  end

  # 4. Aprovisionamiento: Instalamos un servidor web al arrancar
  config.vm.provision "shell", inline: <<-SHELL
```



```
    apt-get update
    apt-get install -y apache2
  SHELL
end
```

6.2 Ejemplo 2: Configuración multi-nodo (Entorno de red aislado)

En arquitectura de sistemas, es habitual separar servicios. Este ejemplo define dos máquinas conectadas por una red privada (Host-Only). El Host y las VMs pueden verse, pero no hay salida directa ni exposición exterior.

```
Vagrant.configure("2") do |config|

  # --- NODO 1: EL SERVIDOR DE APLICACIONES ---
  config.vm.define "app" do |app|
    app.vm.box = "debian/bookworm64"
    app.vm.hostname = "srv-app"
    # Red privada: IP fija interna para que otros nodos le hablen
    app.vm.network "private_network", ip: "192.168.50.10"

    app.vm.provider "virtualbox" do |v|
      v.memory = 512
    end
  end

  # --- NODO 2: EL SERVIDOR DE BASE DE DATOS ---
  config.vm.define "db" do |db|
    db.vm.box = "debian/bookworm64"
    db.vm.hostname = "srv-db"
    db.vm.network "private_network", ip: "192.168.50.11"

    db.vm.provider "virtualbox" do |v|
      v.memory = 1024
    end

    # Aprovisionamiento externo: usamos un script alojado en una carpeta
    db.vm.provision "shell", path: "scripts/install_mysql.sh"
  end
end
```

6.3 Ejemplo 3: Integración con red física (Bridge Dedicado)

Este es el modelo de nuestro laboratorio (Unidad 02). Aquí la máquina virtual sale del aislamiento y se conecta directamente al puente virtual (LabCEFIRE) que creamos en el Host. La VM actúa como un ciudadano de pleno derecho en esa subred, con su propia IP expuesta.

```

Vagrant.configure("2") do |config|

  config.vm.define "webserver" do |web|
    web.vm.box = "debian/bookworm64"
    web.vm.hostname = "ud02-webserver"

    # Conexión directa al Bridge virtual creado en el Host
    # En Vagrant, un Bridge se define usando "public_network"
    web.vm.network "public_network", ip: "172.28.0.20", bridge: "LabCEFIRE"

    web.vm.provider "virtualbox" do |vb|
      vb.name = "UD02-Webserver"
      vb.memory = "1024"

      # Modificación avanzada de hardware:
      # Permite que la tarjeta virtual escuche tráfico promiscuo en el bridge
      vb.customize ["modifyvm", :id, "--nicpromisc2", "allow-all"]
    end

    # Aprovisionamiento modular (Ejecución secuencial de scripts)
    web.vm.provision "shell", path: "scripts/01-update.sh"
    web.vm.provision "shell", path: "scripts/04-install-nginx.sh"
  end
end

```

 **Nota:** Fíjate en la directiva `vb.customize`. Es una "trampilla" que Vagrant nos da para enviarle comandos puros y crudos de VBoxManage a VirtualBox. En este caso, lo usamos para activar el modo promiscuo, vital para que el Bridge procese las tramas de red correctamente.

7. ★ BUENAS PRÁCTICAS Y PRINCIPIOS DE DISEÑO PARA UN VAGRANTFILE EFICIENTE

El Vagrantfile es el corazón de la definición de tu entorno de desarrollo virtualizado. Seguir un conjunto de buenas prácticas no solo facilita la vida del desarrollador individual, sino que también asegura la coherencia, la reproducibilidad y la mantenibilidad de los entornos en un equipo o a lo largo del tiempo. Principios Fundamentales

1. **Idempotencia:** La capacidad de "Re-ejecutar sin daño"

- **Concepto:** Un script de provisioning es idempotente si ejecutarlo una vez produce el mismo resultado que ejecutarlo múltiples veces. En el contexto de Vagrant, esto significa que el comando `vagrant provision` puede ejecutarse en cualquier momento sin estropear una máquina que ya está configurada.
- **Implementación práctica:**
 - Comprobaciones de estado: Antes de instalar un paquete o configurar un servicio, comprueba si ya existe. Por ejemplo, en un script de Shell, usa

condicionales (if [! -f /etc/apache2/apache2.conf]; then ... fi).

- Comandos no interactivos: Utiliza flags como -y en gestores de paquetes (apt-get install -y, yum install -y) o utiliza herramientas de configuración como Ansible o Chef que manejan la idempotencia de forma nativa.

Ejemplo de Shell: En lugar de simplemente instalar un paquete, comprueba su existencia:

```
if ! command -v git &> /dev/null
then
    echo "Git no está instalado. Instalando..."
    sudo apt-get update
    sudo apt-get install -y git
else
    echo "Git ya está instalado. Omitiendo."
fi
```

2. Versionado: El Vagrantfile como plano maestro

- **Concepto:** El Vagrantfile es una descripción textual y ejecutable de tu infraestructura de desarrollo. Al igual que el código fuente de la aplicación, debe ser tratado como un activo crítico y estar bajo control de versiones.
- **Implementación práctica:**
 - **Repositorio central:** Sube siempre el Vagrantfile y todos los scripts/artefactos de provisioning asociados a tu repositorio de control de versiones (Git). Esto garantiza que todo el equipo trabaje con la misma definición de entorno.
 - **Resiliencia y recuperación:** Este versionado actúa como un mecanismo de copia de seguridad infalible. Si el equipo de un desarrollador falla (ej. "si el portátil de un desarrollador explota"), la recuperación es trivial: clonar el repositorio, ejecutar vagrant up y volver a estar operativo en cuestión de minutos, con una réplica exacta del entorno perdido.
 - **Historial y auditoría:** El control de versiones te permite rastrear quién hizo cambios en la definición del entorno, cuándo y por qué, facilitando la depuración de problemas de configuración.

3. Modularidad: Mantén la claridad y la escalabilidad

- **Concepto:** Para proyectos que requieren una configuración extensa (múltiples servicios, bases de datos, herramientas específicas), incrustar todo el código de provisioning dentro del Vagrantfile (como scripts inline) lo convierte en un archivo monolítico, difícil de leer, mantener y depurar.
- **Implementación práctica:**
 - Scripts externos: La mejor práctica es externalizar la lógica de instalación y configuración a scripts separados (ej. bootstrap.sh, install_database.sh).
 - Carpeta de provisioning: Organiza estos scripts en una carpeta dedicada (comúnmente llamada provisioning, scripts o setup).
 - Uso de aprovisionadores: Utiliza los aprovisionadores de Vagrant (Shell,

Ansible, Puppet, Chef, etc.) para llamar a estos archivos externos de forma limpia, haciendo que el Vagrantfile actúe como un índice de configuración.

- **Vagrantfile limpio:** El código final del Vagrantfile permanece limpio y legible, centrando su propósito en la definición de la máquina base y la referencia a la configuración:
`Vagrant.configure("2") do |config|`

4. Consideraciones Adicionales

- **Documentación:** Aunque el Vagrantfile es código como infraestructura, añade comentarios claros y concisos, especialmente en secciones complejas como la configuración de redes o port forwarding.
- **Minimalismo de boxes:** Utiliza boxes base lo más minimalistas posibles (ej. sistemas operativos sin escritorio). Esto reduce el tamaño de las descargas y el tiempo de arranque de la máquina virtual. Instala solo lo que necesites a través de provisioning.
- **Networking:** Define explícitamente la red privada si es necesario (config.vm.network "private_network", ip: "192.168.33.10") para asegurar una dirección IP constante que las aplicaciones puedan referenciar. Evita el port forwarding excesivo si una red privada es suficiente.

8. WEBGRAFÍA

- **HashiCorp Vagrant Documentation**  <https://developer.hashicorp.com/vagrant/docs>
 - Documentación oficial sobre Vagrant, herramienta de HashiCorp para crear y gestionar entornos virtualizados reproducibles mediante archivos de configuración. Incluye instalación, comandos CLI y ejemplos prácticos.
- **Vagrant Cloud (Boxes Registry)**  <https://app.vagrantup.com/boxes/search>
 - Plataforma oficial para buscar y compartir boxes de Vagrant (imágenes base preconfiguradas para distintos sistemas operativos).